Contents

Cyclic Codes

This document is a full English adaptation of the original Korean note on cyclic codes. It preserves the original structure, examples, exercises, and SageMath practice code while translating the exposition into English.

Historical Development of Cyclic Codes

Cyclic codes are tightly intertwined with the broader development of coding theory. The following timeline summarizes major milestones in the history of linear codes and cyclic codes.

1948 - Birth of Coding Theory

Claude Shannon founded information theory by publishing *A Mathematical Theory of Communication* [1]. Through the channel coding theorem, he proved that there exist codes whose error probability can be made arbitrarily small even over noisy channels. However, he did not provide explicit constructions. This result became the starting point for decades of research into how to design "good" codes.

1950 - Hamming Codes

Richard Hamming proposed the linear code $[2^r - 1,\ 2^r - 1 - r]_2$ capable of correcting a single error [2]. This was the first systematic error-correcting code, and for certain parameter choices it has a cyclic structure. The $[7, 4]_2$ code discussed in this document is a representative example.

1954 - Reed-Muller Codes

Irving Reed and David Muller independently introduced Reed-Muller codes, which can correct multiple errors [3][4]. These codes are built from the algebraic structure of Boolean functions, and were famously used in the 1969 Mariner 9 Mars mission to transmit images.

1957 - Algebraic Foundation of Cyclic Codes

Eugene Prange formalized the concept of cyclic codes and established the correspondence with ideals in the polynomial quotient ring $\mathbb{F}_q[X]/\langle X^n - 1\rangle$ [5]. This is the origin of the algebraic viewpoint developed later in this note.

1959-1960 - BCH Codes

Alexis Hocquenghem (1959) [6] and Raj Bose and Dwijendra Ray-Chaudhuri (1960) [7] independently discovered BCH (Bose-Chaudhuri-Hocquenghem) codes. By specifying roots of the generator polynomial over an extension field, they obtained the first systematic design method guaranteeing a target designed distance.

1960 - Reed-Solomon Codes

Irving Reed and Gustave Solomon introduced Reed-Solomon (RS) codes, which are nonbinary cyclic codes [8]. Because of their excellent symbol-level error correction, they have been applied widely in CDs, DVDs, QR codes, and deep-space communications.

1961 - Peterson's Decoding Algorithm

W. Wesley Peterson organized the algebraic decoding procedure for BCH codes and published *Error-Correcting Codes* [9]. It is widely regarded as the first comprehensive textbook on cyclic code theory.

1968 - Berlekamp-Massey Algorithm

Elwyn Berlekamp [10] and James Massey [11] developed an efficient iterative algorithm for computing the error-locator polynomial. This became a key ingredient in practical decoding of BCH and RS codes.

1970s - Industrial Standardization of CRC

CRC (Cyclic Redundancy Check), an error-detection-oriented specialization of cyclic codes, was adopted as an industry standard in Ethernet, communication protocols, and storage devices. The decisive advantage was that CRC can be implemented extremely efficiently in hardware using LFSRs.

1977 - Goppa Codes and the McEliece Cryptosystem

V. D. Goppa laid the foundation of algebraic geometry codes [12], and Robert McEliece proposed the first code-based public-key cryptosystem using Goppa codes [13]. The McEliece scheme is still regarded as quantum-resistant, and it serves as a conceptual ancestor of modern post-quantum code-based cryptography.

Since the 1990s - Modern Developments

With the appearance of turbo codes in 1993 [14] and the rediscovery of LDPC codes in 1996 [15], coding theory shifted toward iterative-decoding, capacity-approaching codes. Even so, cyclic codes remain important because of their hardware efficiency and algebraic clarity, and they still play central roles in flash memories, satellite communications, and post-quantum cryptography.

Shift in Research Perspective

One interesting aspect of the history of cyclic codes is that the direction of research changed over time.

Early stage (1950s): engineering needs -> discovery of algebraic structure

```
"If we build codes that remain valid under cyclic shifts, they can be implemented with LFSRs"
    -> search for linear codes closed under cyclic shift (Prange, 1957)
    -> mathematical analysis reveals equivalence with ideals of F_q[X]/<X^n-1>
    -> existing algebra (ideal theory) becomes immediately applicable
```

In this period, the desired engineering property came first, and the ideal structure was discovered in the process of formalizing it.

Transition period (from 1959): algebraic structure -> code design with target performance

```
"If we prescribe roots of the generator polynomial over an extension field, we can guarantee a
    -> BCH codes (Hocquenghem 1959, Bose-Ray-Chaudhuri 1960)
    -> Reed-Solomon codes (Reed-Solomon 1960)
    -> the designed distance delta can be chosen in advance to control t-error-correction capab
```

From this point on, algebraic structure was actively exploited to design codes with desired error-correcting performance.

Modern stage (since the 1970s): bidirectional interaction

```
algebraic structure <-> engineering applications
    -> CRC: exploit cyclic structure for hardware-efficient error detection
    -> Goppa/McEliece: coding theory -> public-key cryptography
    -> HQC: quasi-cyclic structure -> smaller keys in post-quantum cryptography
```

Today, algebraic structure and engineering applications continue to influence one another in both directions.

What Is a Cyclic Code?

A cyclic code is a special subclass of linear block codes that satisfies two key properties simultaneously: **linearity** and **invariance under cyclic shift**.

A linear code $\mathcal{C}$ of length n is called a cyclic code if it satisfies both of the following:

1. **Linearity**: for any two codewords $\mathbf{c}_1, \mathbf{c}_2 \in \mathcal{C}$, we have $\mathbf{c}_1 + \mathbf{c}_2 \in \mathcal{C}$.
2. **Cyclic shift property**: if $\mathbf{c} = (c_0, c_1, \dots, c_{n-1}) \in \mathcal{C}$, then the vector obtained by a one-step cyclic right shift,
$$\mathbf{c}^{\ggg 1} = (c_{n-1}, c_0, c_1, \dots, c_{n-2}),$$
must also belong to $\mathcal{C}$.

Therefore any cyclic shift by two steps, three steps, or in fact any number of steps also yields a valid codeword.

Example 1.1: Checking a Cyclic Shift

In a binary $[7, 4]_2$ cyclic code, suppose

$$\mathbf{c} = (1, 0, 0, 1, 0, 1, 1)$$

is a codeword. Then:

- 1-step shift: $\mathbf{c}^{\ggg 1} = (1, 1, 0, 0, 1, 0, 1)$ -> codeword
- 2-step shift: $\mathbf{c}^{\ggg 2} = (1, 1, 1, 0, 0, 1, 0)$ -> codeword
- 3-step shift: $\mathbf{c}^{\ggg 3} = (0, 1, 1, 1, 0, 0, 1)$ -> codeword

All of these vectors must belong to the code space $\mathcal{C}$.

Example 1.2: Deciding Whether a Code Is Cyclic

Let $n = 4$ and consider the set

$$\mathcal{C} = \{(0, 0, 0, 0),\ (1, 0, 1, 0),\ (0, 1, 0, 1),\ (1, 1, 1, 1)\}.$$

- Linearity check: $(1,0,1,0) \oplus (0,1,0,1) = (1,1,1,1) \in \mathcal{C}$.
- Cyclic-shift check:
  - $(1,0,1,0)$ shifted once gives $(0,1,0,1) \in \mathcal{C}$.
  - $(0,1,0,1)$ shifted once gives $(1,0,1,0) \in \mathcal{C}$.
  - $(1,1,1,1)$ shifted once gives $(1,1,1,1) \in \mathcal{C}$.

Hence this is a cyclic code.

Example 1.3: A Linear Code That Is Not Cyclic

Let

$$\mathcal{C}' = \{(0,0,0,0),\ (1,1,0,0),\ (0,0,1,1),\ (1,1,1,1)\}.$$

Shifting $(1,1,0,0)$ one step cyclically gives $(0,1,1,0)$, which is not in $\mathcal{C}'$. Therefore $\mathcal{C}'$ is linear, but **not** cyclic.

Why Is Cyclic-Shift Invariance Useful for Implementation?

Cyclic codes were originally studied not because of abstract algebra, but because of **hardware efficiency**. Prange's key observation in the 1950s was that if a code is closed under cyclic shift, then encoding and error detection can be implemented using nothing more than shift registers.

The Drawback of General Linear Codes

For a general linear $[n,k]_q$ code, encoding is done by matrix multiplication:

$$\mathbf{c} = \mathbf{m} \cdot G.$$

This means:

- the $k \times n$ generator matrix G must be stored in memory,
- $k \cdot n$ additions and multiplications are needed,
- computation cannot begin until the whole message block has arrived.

For example, for a $[31,21]_2$ code, one must store $21 \times 31 = 651$ matrix entries and perform 651 binary operations for every block.

The Cyclic-Code Alternative: Shift Registers

For cyclic codes, every codeword is a multiple of the generator polynomial $g(X)$, and encoding reduces to polynomial multiplication or polynomial division with remainder. These operations can be implemented using an LFSR (linear feedback shift register):

- only $r = n - k$ one-bit registers are required,
- the input stream is processed one bit per clock,
- after the last bit passes through the circuit, the register contents are exactly the parity bits or syndrome.

For a $[31,21]_2$ code, 10 registers plus a few XOR gates are enough.

Example: Matrix Multiplication vs. LFSR for a $[7,4]_2$ Code

Item	General linear code (matrix multiplication)	Cyclic code (LFSR)
Storage	$G \in GF(2)^{4\times 7}$ -> 28 bits	$g(X) = 1 + X + X^3$ -> 4 bits (coefficients $[1, 1, 0, 1]$)
Operations per block	$4 \times 7 = 28$ AND/XOR operations	7 clocks (roughly one XOR per bit)
Processing mode	Batch processing after full block arrival	Real-time, bit-by-bit
Latency	Proportional to block length	Almost zero
Circuit complexity	n XOR trees driven by k inputs	r flip-flops plus XOR gates

Detailed Analysis: Why Does It Take 7 Clocks? (Non-Systematic Encoding)

Non-systematic encoding $c(X) = m(X)g(X)$ is, mathematically, a convolution of two polynomials. For a $[7, 4]_2$ code the message has 4 bits, the generator polynomial has degree 3, and the codeword has length 7.

1. Coefficient Expansion of the Polynomial Product Let

$$m(X) = m_0 + m_1X + m_2X^2 + m_3X^3,$$

$$g(X) = g_0 + g_1X + g_2X^2 + g_3X^3.$$

Then in

$$c(X) = \sum_{k=0}^{6} c_kX^k,$$

we have:

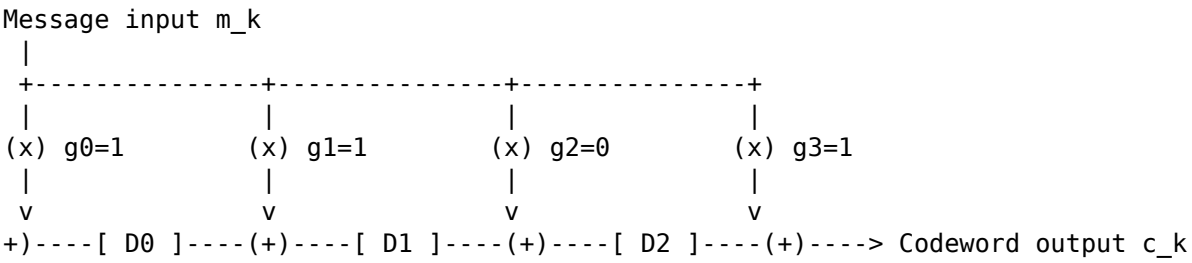
- $c_0 = m_0g_0$
- $c_1 = m_0g_1 + m_1g_0$
- $c_2 = m_0g_2 + m_1g_1 + m_2g_0$
- $c_3 = m_0g_3 + m_1g_2 + m_2g_1 + m_3g_0$
- $c_4 = m_1g_3 + m_2g_2 + m_3g_1$
- $c_5 = m_2g_3 + m_3g_2$
- $c_6 = m_3g_3$

2. Clock-by-Clock Operation (Serial Processing) In a serial implementation, one message bit enters per clock, and one codeword bit is emitted per clock.

Clock	Input	Output	Explanation
1	m_0	c_0	The first codeword bit appears immediately
2	m_1	c_1	Contributions of m_0 and m_1 combine
3	m_2	c_2	

Clock	Input	Output	Explanation
4	m_3	c_3	Message input finishes, but the tail has not flushed out yet
5	0	c_4	Feed zeros to flush the remaining state
6	0	c_5	
7	0	c_6	Final output bit

3. Hardware Structure (ASCII Art) For $g(X) = 1 + X + X^3$:



How it works: 1. **Input phase (clocks 1-4):** message bits m_0, m_1, m_2, m_3 are fed in sequence. At each clock the current input bit is multiplied by each coefficient of $g(X)$ and combined with delayed values stored in the registers. 2. **Flush phase (clocks 5-7):** after the message bits are exhausted, zeros are shifted in. The intermediate values stored in D_0, D_1, D_2 are flushed out, yielding c_4, c_5, c_6. 3. **Result:** after 7 clocks all registers are cleared and the 7-bit codeword has been fully emitted.

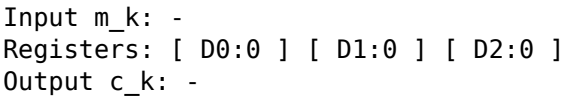
Hence the total number of clocks is

$$4 + 3 = 7,$$

which equals the codeword length n.

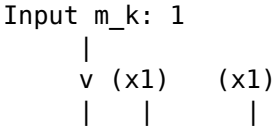
4. Step-by-Step Visualization Let the message be $\mathbf{m} = (1, 1, 0, 1)$ and let $g(X) = 1 + X + X^3$.

Initial state: all registers are zero.



Clock 1: input $m_0 = 1$

- $c_0 = 1 \cdot 1 \oplus 0 = 1$
- New state: $D_0 \leftarrow 1$, $D_1 \leftarrow 0$, $D_2 \leftarrow 1$



```
0 --(+)--[ D0 ]--(+)--[ D1 ]--(+)--[ D2 ]--(+)--> Output c_0: 1
         (New:1)      (New:0)      (New:1)
```

Clock 2: input $m_1 = 1$

- $c_1 = 1 \cdot 1 \oplus D_0 = 1 \oplus 1 = 0$
- New state: $D_0 \leftarrow 1$, $D_1 \leftarrow 1$, $D_2 \leftarrow 1$

```
Input m_k: 1
     |
     v
0 --(+)--[ D0:1 ]--(+)--[ D1:0 ]--(+)--[ D2:1 ]--(+)--> Output c_1: 0
         (New:1)        (New:1)        (New:1)
```

Clock 3: input $m_2 = 0$

- $c_2 = 0 \cdot 1 \oplus D_0 = 1$
- New state: $D_0 \leftarrow 1$, $D_1 \leftarrow 1$, $D_2 \leftarrow 0$

```
Input m_k: 0
     |
     v
0 --(+)--[ D0:1 ]--(+)--[ D1:1 ]--(+)--[ D2:1 ]--(+)--> Output c_2: 1
         (New:1)        (New:1)        (New:0)
```

Clock 4: input $m_3 = 1$

- $c_3 = 1 \cdot 1 \oplus D_0 = 0$
- New state: $D_0 \leftarrow 0$, $D_1 \leftarrow 0$, $D_2 \leftarrow 1$

```
Input m_k: 1
     |
     v
0 --(+)--[ D0:1 ]--(+)--[ D1:1 ]--(+)--[ D2:0 ]--(+)--> Output c_3: 0
         (New:0)        (New:0)        (New:1)
```

Clock 5: flush with input 0

- $c_4 = 0 \cdot g_0 \oplus D_0 = 0$
- New state: $D_0 \leftarrow 0$, $D_1 \leftarrow 1$, $D_2 \leftarrow 0$

```
Input m_k: 0
     |
     v
0 --(+)--[ D0:0 ]--(+)--[ D1:0 ]--(+)--[ D2:1 ]--(+)--> Output c_4: 0
         (New:0)        (New:1)        (New:0)
```

Clock 6: flush again

- $c_5 = 0 \cdot g_0 \oplus D_0 = 0$
- New state: $D_0 \leftarrow 1$, $D_1 \leftarrow 0$, $D_2 \leftarrow 0$

```
Input m_k: 0
     |
     v
0 --(+)--[ D0:0 ]--(+)--[ D1:1 ]--(+)--[ D2:0 ]--(+)--> Output c_5: 0
         (New:1)        (New:0)        (New:0)
```

Clock 7: final output

- $c_6 = 0 \cdot g_0 \oplus D_0 = 1$
- All registers return to zero.

```
Input m_k: 0
     |
     v
0 --(+)--[ D0:1 ]--(+)--[ D1:0 ]--(+)--[ D2:0 ]--(+)--> Output c_6: 1
         (New:0)        (New:0)        (New:0)
```

Final result:

$$\mathbf{c} = (1, 0, 1, 0, 0, 0, 1).$$

This matches the polynomial product

$$c(X) = (1 + X + X^3)(1 + X + X^3) = 1 + X^2 + X^6.$$

Why Does "Cyclic" Enable Shift Registers?

The key correspondence is

$$\text{cyclic shift} \longleftrightarrow X \cdot c(X) \pmod{X^n - 1}.$$

The basic action of a shift register - moving all bits by one position and injecting feedback - is exactly multiplication by X followed by reduction modulo $X^n - 1$. This is why cyclic codes admit LFSR implementations, whereas general linear codes do not.

In summary:

```
cyclic-shift invariance
    -> codewords are multiples of g(X)
    -> encoding becomes polynomial division / multiplication
    -> implementable by LFSR
    -> storage O(n-k), computation O(n), real-time processing
```

This is the fundamental reason cyclic codes became so important, and why CRC remains the standard error-detection mechanism in protocols such as Ethernet and USB.

How Do We Represent Cyclic Codes Using Polynomials?

A vector of length n,

$$\mathbf{c} = (c_0, c_1, \dots, c_{n-1}),$$

can be mapped to a polynomial of degree at most $n - 1$:

$$c(X) = c_0 + c_1 X + c_2 X^2 + \cdots + c_{n-1} X^{n-1}.$$

Under this representation, a one-step cyclic right shift corresponds exactly to

$$X \cdot c(X) \pmod{X^n - 1}.$$

Therefore all algebraic operations on a cyclic code naturally live in the quotient ring

$$R_n = \mathbb{F}_q[X]/\langle X^n - 1 \rangle.$$

Example 2.1: Vector <-> Polynomial Conversion

For $n = 7$:

Vector	Polynomial
$(1,0,1,1,0,0,0)$	$1 + X^2 + X^3$
$(0,1,0,0,1,1,0)$	$X + X^4 + X^5$
$(1,1,1,1,1,1,1)$	$1 + X + X^2 + X^3 + X^4 + X^5 + X^6$

Example 2.2: Cyclic Shift as a Polynomial Operation

Let $n = 7$ and $c(X) = 1 + X^2 + X^3$. Then

$$Xc(X) = X + X^3 + X^4.$$

Since the degree is still below 7, reduction modulo $X^7 - 1$ changes nothing. The corresponding vector is $(0,1,0,1,1,0,0)$, which is exactly the one-step cyclic shift of $(1,0,1,1,0,0,0)$.

Example 2.3: When the Highest-Degree Term Wraps Around

Let $n = 7$ and $c(X) = X^4 + X^5 + X^6$. Then

$$Xc(X) = X^5 + X^6 + X^7.$$

Since $X^7 \equiv 1 \pmod{X^7 - 1}$,

$$Xc(X) \bmod (X^7 - 1) = 1 + X^5 + X^6.$$

Thus

$$(0,0,0,0,1,1,1) \xrightarrow{\text{1-step shift}} (1,0,0,0,0,1,1).$$

What Is a Generator Polynomial, and What Properties Does It Have?

Every cyclic code is completely determined by a unique monic polynomial $g(X)$ of **smallest degree** in the code. This polynomial is called the **generator polynomial**.

Key properties:

- $g(X)$ must divide $X^n - 1$:
$$X^n - 1 = g(X)h(X).$$
- If $\deg g = r = n - k$, then the code dimension is k.
- Every codeword is a multiple of $g(X)$:
$$c(X) = m(X)g(X).$$

For **non-systematic encoding**, one simply multiplies the message polynomial by the generator polynomial:

$$c(X) = m(X)g(X).$$

Example 3.1: Factorization of $X^7 - 1$ and Possible Generator Polynomials

Over $GF(2)$, we have $-1 = 1$, so $X^7 - 1 = X^7 + 1$, and it factors as

$$X^7 + 1 = (1 + X)(1 + X + X^3)(1 + X^2 + X^3).$$

Any divisor of this polynomial can serve as a generator polynomial:

$g(X)$	$\deg(g) = n-k$	$[n, k]_2$	Number of codewords 2^k
$1 + X$	1	$[7, 6]_2$	64
$1 + X + X^3$	3	$[7, 4]_2$	16
$1 + X^2 + X^3$	3	$[7, 4]_2$	16
$(1 + X)(1 + X + X^3) = 1 + X^2 + X^3 + X^4$	4	$[7, 3]_2$	8
$(1 + X)(1 + X^2 + X^3) = 1 + X + X^2 + X^4$	4	$[7, 3]_2$	8
$(1 + X + X^3)(1 + X^2 + X^3) = 1 + X + X^2 + X^3 + X^4 + X^5 + X^6$	6	$[7, 1]_2$	2
$X^7 + 1$	7	$[7, 0]_2$	1 (trivial)

Example 3.2: Non-Systematic Encoding for a $[7, 4]_2$ Code

Let $g(X) = 1 + X + X^3$ and let the message be

$$\mathbf{m} = (1, 1, 0, 1), \qquad m(X) = 1 + X + X^3.$$

Then

$$\begin{aligned} c(X) &= m(X)g(X) \\ &= (1 + X + X^3)(1 + X + X^3) \\ &= 1 + X + X^3 + X + X^2 + X^4 + X^3 + X^4 + X^6 \\ &= 1 + (1 + 1)X + X^2 + (1 + 1)X^3 + (1 + 1)X^4 + X^6 \\ &= 1 + X^2 + X^6. \end{aligned}$$

Thus the codeword vector is

$$\mathbf{c} = (1, 0, 1, 0, 0, 0, 1).$$

Example 3.3: Another Non-Systematic Encoding Example

With the same generator polynomial, let the message be

$$\mathbf{m} = (0, 1, 0, 0), \qquad m(X) = X.$$

Then

$$c(X) = X(1 + X + X^3) = X + X^2 + X^4.$$

So the codeword is

$$\mathbf{c} = (0, 1, 1, 0, 1, 0, 0).$$

This is exactly the coefficient vector of $Xg(X)$, i.e. the second row of the generator matrix.

What Is a Parity-Check Polynomial?

The quotient obtained by dividing $X^n - 1$ by the generator polynomial is called the **parity-check polynomial**:

$$h(X) = \frac{X^n - 1}{g(X)}.$$

Its degree is k. To check whether a polynomial $v(X)$ is a valid codeword, one tests

$$v(X)h(X) \equiv 0 \pmod{X^n - 1}.$$

Example 4.1: Parity-Check Polynomial for a $[7, 4]_2$ Code

If $g(X) = 1 + X + X^3$, then

$$h(X) = \frac{X^7 + 1}{1 + X + X^3} = 1 + X + X^2 + X^4.$$

Verification:

$$g(X)h(X) = (1 + X + X^3)(1 + X + X^2 + X^4) = X^7 + 1.$$

Indeed, term-by-term cancellation in $GF(2)$ leaves only $1 + X^7$.

Example 4.2: Validating a Codeword

Consider the codeword

$$c(X) = X + X^2 + X^4.$$

Then

$$c(X)h(X) = (X + X^2 + X^4)(1 + X + X^2 + X^4).$$

Expanding and reducing modulo $X^7 + 1$ yields 0, so $c(X)$ is a valid codeword.

How Do We Construct a Generator Matrix?

Let

$$g(X) = g_0 + g_1 X + \cdots + g_{n-k} X^{n-k}.$$

Then the generator matrix G is built by taking the coefficient vector of $g(X)$ and shifting it one position to the right in each row:

$$G = \begin{bmatrix} g_0 & g_1 & g_2 & \cdots & g_{n-k} & 0 & \cdots & 0 \\ 0 & g_0 & g_1 & \cdots & g_{n-k-1} & g_{n-k} & \cdots & 0 \\ \vdots & & \ddots & & & & \ddots & \vdots \\ 0 & \cdots & 0 & g_0 & g_1 & \cdots & \cdots & g_{n-k} \end{bmatrix}.$$

The i-th row corresponds to the coefficient vector of $X^i g(X)$.

> **Why is this a valid generator matrix?** 1. $g(X)$ is a codeword, and by cyclic-shift invariance so are $Xg(X), X^2 g(X), \ldots, X^{k-1} g(X)$. 2. These vectors are linearly independent because their highest nonzero terms occur in different positions. 3. A k-dimensional code containing k linearly independent codewords has found a basis.

Example 5.1: Generator Matrix for a $[7,4]_2$ Cyclic Code

Let $g(X) = 1 + X + X^3$, whose coefficient vector is $(1,1,0,1)$. Then

$$G = \begin{bmatrix} 1 & 1 & 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 & 0 & 1 \end{bmatrix}.$$

Interpretation of the rows:

- Row 0: $g(X) = 1 + X + X^3$
- Row 1: $Xg(X) = X + X^2 + X^4$
- Row 2: $X^2 g(X) = X^2 + X^3 + X^5$
- Row 3: $X^3 g(X) = X^3 + X^4 + X^6$

Example 5.2: Encoding with the Generator Matrix

Let $\mathbf{m} = (1,0,1,1)$. Then

$$\mathbf{c} = \mathbf{m}G = (1,0,1,1) \begin{bmatrix} 1 & 1 & 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 & 0 & 1 \end{bmatrix}.$$

Carrying out the multiplication over $GF(2)$ gives

$$\mathbf{c} = (1,1,1,1,1,1,1).$$

Thus

$$c(X) = 1 + X + X^2 + X^3 + X^4 + X^5 + X^6,$$

and this is divisible by $g(X)$.

Example 5.3: A $(7,3)$ Cyclic Code

Let $g(X) = 1 + X^2 + X^3 + X^4$. Then $k = 3$ and

$$G = \begin{bmatrix} 1 & 0 & 1 & 1 & 1 & 0 & 0 \\ 0 & 1 & 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 & 1 & 1 & 1 \end{bmatrix}.$$

For message $\mathbf{m} = (1,1,0)$,

$$\mathbf{c} = (1,1,0)G = (1,1,1,0,0,1,0).$$

How Do We Construct a Parity-Check Matrix?

Let

$$h(X) = h_0 + h_1 X + \cdots + h_k X^k.$$

To construct the parity-check matrix H, reverse the coefficient order of $h(X)$ and shift that reversed vector across the rows:

$$H = \begin{bmatrix} h_k & h_{k-1} & \dots & h_0 & 0 & \dots & 0 \\ 0 & h_k & h_{k-1} & \dots & h_0 & \dots & 0 \\ \vdots & & \ddots & & & \ddots & \vdots \\ 0 & \dots & 0 & h_k & h_{k-1} & \dots & h_0 \end{bmatrix}.$$

This matrix satisfies the orthogonality relation

$$GH^T = 0.$$

Why reverse the coefficients?

Since $g(X)h(X) = X^n - 1 \equiv 0 \pmod{X^n - 1}$ and every codeword has the form $c(X) = m(X)g(X)$, we have

$$c(X)h(X) \equiv 0.$$

Writing this condition as inner products of coefficient vectors leads precisely to the reversed-coefficient construction of H.

Example 6.1: Parity-Check Matrix for a $[7, 4]_2$ Cyclic Code

For

$$h(X) = 1 + X + X^2 + X^4,$$

its coefficient vector is $(1, 1, 1, 0, 1)$ and the reversed vector is $(1, 0, 1, 1, 1)$. Therefore

$$H = \begin{bmatrix} 1 & 0 & 1 & 1 & 1 & 0 & 0 \\ 0 & 1 & 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 & 1 & 1 & 1 \end{bmatrix}.$$

Example 6.2: Verifying Orthogonality

Take the first row of G, $(1, 1, 0, 1, 0, 0, 0)$, and the first row of H, $(1, 0, 1, 1, 1, 0, 0)$. Their inner product is

$$1 \cdot 1 + 1 \cdot 0 + 0 \cdot 1 + 1 \cdot 1 = 1 + 1 = 0 \pmod 2.$$

Likewise, the first row of G is orthogonal to the second and third rows of H, and similarly for all remaining rows.

Example 6.3: Error Detection with the Parity-Check Matrix

For

$$\mathbf{v} = (1, 1, 1, 1, 1, 1, 1),$$

we get

$$H\mathbf{v}^T = \mathbf{0},$$

so no error is detected.

If instead

$$\mathbf{v}' = (0, 1, 1, 1, 1, 1, 1),$$

then

$$H\mathbf{v}'^T = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} \neq \mathbf{0},$$

so the error is detected.

What Is the Difference Between Non-Systematic and Systematic Encoding?

Both methods produce valid codewords, but they place the original message differently inside the codeword.

Category	Non-systematic	Systematic
Encoding rule	$c(X) = m(X)g(X)$	$c(X) = p(X) + X^{n-k}m(X)$
Message position	Mixed throughout the codeword	Preserved explicitly in fixed positions
Ease of decoding	Requires division	Message bits can be read directly
Typical practical use	Rare	Common in real systems

Example 7.1: Two Encodings of the Same Message

Let $g(X) = 1 + X + X^3$ and let the message be $\mathbf{m} = (1, 0, 1, 1)$.

Non-systematic encoding:

$$c(X) = m(X)g(X) = (1 + X^2 + X^3)(1 + X + X^3) = 1 + X + X^2 + X^3 + X^4 + X^5 + X^6.$$

The codeword is $(1, 1, 1, 1, 1, 1, 1)$, and the message is not visibly preserved as a contiguous block.

Systematic encoding: the resulting codeword has the form

$$(1, 0, 0, \mathbf{1}, \mathbf{0}, \mathbf{1}, \mathbf{1}),$$

so the last four bits are exactly the original message.

How Is Systematic Encoding Performed in Practice?

Systematic encoding proceeds in three steps.

Step 1. Shift the message polynomial

$$X^{n-k}m(X).$$

Step 2. Compute the remainder

$$p(X) = X^{n-k}m(X) \bmod g(X).$$

Step 3. Form the codeword

$$c(X) = p(X) + X^{n-k}m(X).$$

Thus the codeword has the structure

$$[\text{parity } (n-k) \text{ bits} \mid \text{message } k \text{ bits}].$$

Why is this a valid codeword? Since $p(X)$ is the remainder of $X^{n-k}m(X)$ modulo $g(X)$,

$$X^{n-k}m(X) - p(X)$$

is divisible by $g(X)$. Over $GF(2)$, subtraction equals addition, so $c(X)$ is also divisible by $g(X)$.

Example 8.1: Systematic Encoding of $(1, 0, 1, 1)$

Let $g(X) = 1 + X + X^3$, $n - k = 3$, and $m(X) = 1 + X^2 + X^3$.

Step 1:

$$X^3 m(X) = X^3 + X^5 + X^6.$$

Step 2: Divide $X^6 + X^5 + X^3$ by $X^3 + X + 1$.

Step	Dividend	Quotient term
1	$X^6 + X^5 + X^3$ $-(X^6 + X^4 + X^3)$ $= X^5 + X^4$	X^3
2	$X^5 + X^4$ $-(X^5 + X^3 + X^2)$ $= X^4 + X^3 + X^2$	X^2
3	$X^4 + X^3 + X^2$ $-(X^4 + X^2 + X)$ $= X^3 + X$	X
4	$X^3 + X$ $-(X^3 + X + 1)$ $= 1$	1

So the quotient is $X^3 + X^2 + X + 1$, and the remainder is

$$p(X) = 1.$$

Step 3:

$$c(X) = 1 + X^3 + X^5 + X^6.$$

Thus

$$\mathbf{c} = (\underbrace{1, 0, 0}_{\text{parity}}, \underbrace{1, 0, 1, 1}_{\text{message}}).$$

Example 8.2: Systematic Encoding of $(1, 1, 0, 0)$

Here $m(X) = 1 + X$.

Step 1:

$$X^3 m(X) = X^3 + X^4.$$

Step 2: Divide by $X^3 + X + 1$ to obtain remainder

$$p(X) = X^2 + 1,$$

which corresponds to parity vector $(1, 0, 1)$.

Step 3:

$$c(X) = 1 + X^2 + X^3 + X^4.$$

So

$$\mathbf{c} = (\underbrace{1, 0, 1}_{\text{parity}}, \ \underbrace{1, 1, 0, 0}_{\text{message}}).$$

Example 8.3: Systematic Encoding of $(0, 0, 0, 1)$

Here $m(X) = X^3$.

Step 1:

$$X^3 m(X) = X^6.$$

Step 2: Dividing by $X^3 + X + 1$ again yields

$$p(X) = X^2 + 1,$$

so the parity bits are $(1, 0, 1)$.

Step 3:

$$c(X) = 1 + X^2 + X^6.$$

Hence

$$\mathbf{c} = (\underbrace{1, 0, 1}_{\text{parity}}, \ \underbrace{0, 0, 0, 1}_{\text{message}}).$$

Example 8.4: Full Table of Systematic Codewords for the $[7, 4]_2$ **Code**

Message $(m_0 m_1 m_2 m_3)$	$m(X)$	Parity $(p_0 p_1 p_2)$	Codeword $(p_0 p_1 p_2 m_0 m_1 m_2 m_3)$
0000	0	000	0000000
1000	1	110	1101000
0100	X	011	0110100
1100	$1 + X$	101	1011100
0010	X^2	111	1110010
1010	$1 + X^2$	001	0011010
0110	$X + X^2$	100	1000110
1110	$1 + X + X^2$	010	0101110
0001	X^3	101	1010001
1001	$1 + X^3$	011	0111001
0101	$X + X^3$	110	1100101
1101	$1 + X + X^3$	000	0001101
0011	$X^2 + X^3$	010	0100011
1011	$1 + X^2 + X^3$	100	1001011
0111	$X + X^2 + X^3$	001	0010111
1111	$1 + X + X^2 + X^3$	111	1111111

What Is the Syndrome, and How Is Error Detection Performed?

The **remainder** obtained by dividing the received polynomial $v(X)$ by the generator polynomial $g(X)$ is called the **syndrome**:

$$s(X) = v(X) \bmod g(X).$$

- If $s(X) = 0$, no error is detected.
- If $s(X) \neq 0$, an error is detected.

  Core principle: if $v(X) = c(X) + e(X)$ and $c(X)$ is a multiple of $g(X)$, then

  $$s(X) = v(X) \bmod g(X) = e(X) \bmod g(X).$$

  Thus the syndrome depends only on the error pattern, not on the original message.

Example 9.1: Error-Free Reception

Suppose the codeword

$$c(X) = 1 + X^3 + X^5 + X^6$$

is received without error. Dividing by $X^3 + X + 1$ yields remainder 0.

Step	Dividend	Quotient term
1	$X^6 + X^5 + X^3 + 1$	X^3
	$-(X^6 + X^4 + X^3)$	
	$= X^5 + X^4 + 1$	
2	$X^5 + X^4 + 1$	X^2
	$-(X^5 + X^3 + X^2)$	
	$= X^4 + X^3 + X^2 + 1$	
3	$X^4 + X^3 + X^2 + 1$	X
	$-(X^4 + X^2 + X)$	
	$= X^3 + X + 1$	
4	$X^3 + X + 1$	1
	$-(X^3 + X + 1)$	
	$= 0$	

So the syndrome is 0.

Example 9.2: A Single-Bit Error at Position X^2

Suppose the received polynomial is

$$v(X) = 1 + X^2 + X^3 + X^5 + X^6,$$

so the error polynomial is $e(X) = X^2$.

Then

$$s(X) = X^2,$$

whose vector representation is $(0, 0, 1)$. Since the syndrome is nonzero, the error is detected.

Example 9.3: Complete Syndrome Table for Single-Bit Errors in the $[7,4]_2$ Code

For a single-bit error $e(X) = X^i$, the syndrome is $s(X) = X^i \bmod g(X)$.

Error position i	$e(X)$	$s(X) = e(X) \bmod (1 + X + X^3)$	Syndrome vector
0	1	1	$(1,0,0)$
1	X	X	$(0,1,0)$
2	X^2	X^2	$(0,0,1)$
3	X^3	$1+X$	$(1,1,0)$
4	X^4	$X+X^2$	$(0,1,1)$
5	X^5	$1+X+X^2$	$(1,1,1)$
6	X^6	$1+X^2$	$(1,0,1)$

Since all 7 syndromes are distinct, the code can correct **any single-bit error**.

Example 9.4: Error Correction Using the Syndrome

Suppose the received vector is

$$\mathbf{v} = (1,0,1,1,0,1,1).$$

Step 1. Compute the syndrome: with

$$v(X) = 1 + X^2 + X^3 + X^5 + X^6,$$

we obtain

$$s(X) = X^2,$$

so the syndrome vector is $(0,0,1)$.

Step 2. Look up the syndrome table: $(0,0,1)$ corresponds to error position $i = 2$.

Step 3. Correct the error:

$$\hat{c}(X) = v(X) + X^2 = 1 + X^3 + X^5 + X^6.$$

Therefore the corrected codeword is

$$\hat{\mathbf{c}} = (1,0,0,1,0,1,1).$$

Step 4. Extract the message: for a systematic code, the last four bits are the message,

$$\mathbf{m} = (1,0,1,1).$$

Example 9.5: The Case of a Two-Bit Error

Suppose the received vector is

$$\mathbf{v} = (0,1,0,1,0,1,1),$$

which corresponds to simultaneous errors at positions 0 and 1. Then

$$e(X) = 1 + X,$$

and the syndrome is

$$s(X) = 1 + X,$$

with vector $(1, 1, 0)$.

But $(1, 1, 0)$ corresponds in the syndrome table to a *single* error at position 3. A decoder designed only for single-error correction will therefore flip the wrong bit, causing a **miscorrection**.

This is exactly the limitation of the $[7, 4]_2$ code: since its minimum distance is $d_{\min} = 3$, it can correct 1 bit or detect 2 bits, but it cannot reliably correct arbitrary 2-bit errors.

What Is the Decoding Process for a Cyclic Code?

The details of decoding depend on the specific family of cyclic code (simple cyclic code, BCH code, RS code, and so on), but the overall structure is usually the same.

Step 1: Syndrome Calculation

Given the received polynomial

$$v(X) = c(X) + e(X),$$

compute

$$s(X) = v(X) \bmod g(X).$$

This can be done very efficiently in hardware by feeding the received stream into an LFSR.

Step 2: Error-Pattern Search

Using the syndrome $s(X)$, determine the actual error polynomial $e(X)$.

- **Syndrome lookup table:** for short block codes (such as small Hamming codes or Golay codes), the syndrome can be mapped directly to an error pattern.
- **Meggitt decoder:** especially hardware-friendly. The syndrome register is shifted one clock at a time while logic checks whether the current pattern indicates an error in the most significant position.
- **Algebraic decoding:** for BCH and Reed-Solomon codes, lookup tables are impractical. One instead computes the error-locator polynomial using methods such as the Berlekamp-Massey algorithm, locates roots by Chien search, and recovers magnitudes via the Forney algorithm.

Step 3: Error Correction

Once an estimated error pattern $\hat{e}(X)$ has been found, subtract it from the received vector:

$$\hat{c}(X) = v(X) - \hat{e}(X).$$

For binary codes, subtraction and addition are the same, so this is just XOR.

Step 4: Message Extraction

Finally, recover the original message.

- **Systematic codes:** simply read the designated message positions in the codeword.
- **Non-systematic codes:** divide the corrected codeword by $g(X)$ and take the quotient.

How Is the Error-Correcting Capability of a Cyclic Code Determined?

A cyclic code is still a linear code, so its error-correcting capability is determined by its **minimum distance** $d_{\min}$. There is no special correction formula unique to cyclic codes; the same general relationship for linear codes applies:

Core Result: t-Error-Correcting Capability

A code with minimum distance $d_{\min}$ can correct up to

$$t = \left\lfloor \frac{d_{\min} - 1}{2} \right\rfloor$$

errors.

If one is interested only in error **detection**, then up to $d_{\min} - 1$ errors can be detected.

Example 11.1: Error-Correcting Capability of Representative Cyclic Codes

Code	$[n, k]$	$d_{\min}$	Correction capability t	Detection capability
$[7, 4]$ cyclic code (Hamming)	$[7, 4]_2$	3	1	2
$[15, 7]$ BCH code	$[15, 7]$	5	2	4
$[23, 12]$ Golay code	$[23, 12]$	7	3	6
$[15, 5]$ BCH code	$[15, 5]$	7	3	6
$[31, 21]$ BCH code	$[31, 21]$	5	2	4

Limitation of a General Cyclic Code

For a general cyclic code, there is usually no simple closed-form formula that gives $d_{\min}$ directly from $g(X)$. One must compute it either by enumerating codewords or by a deeper algebraic analysis.

BCH Bound: Guaranteed Designed Distance

A famous theorem does exist for the important subclass of **BCH codes**.

> **BCH bound:** Let α be a primitive nth root of unity satisfying $X^n - 1 = 0$ in $GF(q^m)$. If the generator polynomial $g(X)$ has the consecutive roots
>
> $$\alpha^b,\ \alpha^{b+1},\ \ldots,\ \alpha^{b+\delta-2},$$

then the minimum distance satisfies

$$d_{\min} \geq \delta.$$

The parameter δ is called the **designed distance**. Because of this theorem, one can choose a target correction capability t in advance by setting

$$\delta = 2t + 1.$$

Example 11.2: Applying the BCH Bound

For the binary $(15, 7)$ BCH code, let α be a primitive 15th root of unity in $GF(2^4)$. If the generator polynomial has roots

$$\alpha, \alpha^2, \alpha^3, \alpha^4,$$

then there are 4 consecutive roots, so

$$\delta = 4 + 1 = 5.$$

Therefore

$$d_{\min} \geq 5 \quad \Rightarrow \quad t \geq 2.$$

In fact, this code attains $d_{\min} = 5$, so the BCH bound is tight here.

Example 11.3: General Cyclic Code vs. BCH Code

For the $[7, 4]_2$ cyclic code with

$$g(X) = 1 + X + X^3,$$

let α be a primitive root of $X^7 + 1$ in $GF(2^3)$. Since

$$g(\alpha) = 0, \qquad g(\alpha^2) = 0,$$

there are two consecutive roots. Hence the BCH bound gives

$$\delta = 3 \quad \Rightarrow \quad d_{\min} \geq 3.$$

The actual minimum distance is exactly 3. In fact this code is both the $[7, 4]_2$ Hamming code and a BCH code.

This ability to design error-correcting performance at the construction stage is one of the main reasons BCH and Reed-Solomon codes are favored in practice over arbitrary cyclic codes.

What Is the Algebraic Structure of a Cyclic Code?

A cyclic code is a **principal ideal** of the quotient ring

$$R_n = \mathbb{F}_q[X]/\langle X^n - 1\rangle.$$

This means two things:

1. The cyclic code $\mathcal{C}$ is an ideal of the ring R_n.
2. This ideal is generated by a single element $g(X)$:

$$\mathcal{C} = \langle g(X)\rangle.$$

Proof Sketch

Step 1: Show that $\mathcal{C}$ is an ideal.

- **Closed under addition:** by linearity, if $a(X), b(X) \in \mathcal{C}$ then $a(X) + b(X) \in \mathcal{C}$.
- **Absorption:** for any $r(X) \in R_n$ and $c(X) \in \mathcal{C}$,

$$r(X)c(X) \in \mathcal{C}.$$

This follows because cyclic-shift invariance implies $Xc(X) \in \mathcal{C}$, repeated shifts give $X^2c(X), X^3c(X), ...$, and linearity then gives closure under any polynomial multiple.

Step 2: Show that the ideal is principal.

- Let $g(X)$ be the monic polynomial of smallest degree in $\mathcal{C}$.
- For any $c(X) \in \mathcal{C}$, divide:

$$c(X) = q(X)g(X) + r(X), \qquad \deg r < \deg g.$$

- Since $r(X) = c(X) - q(X)g(X) \in \mathcal{C}$ and $g(X)$ has minimal degree, we must have $r(X) = 0$.
- Hence every codeword is a multiple of $g(X)$, so

$$\mathcal{C} = \langle g(X) \rangle.$$

Example 10.1: Checking the Ideal Structure

For the $[7, 4]_2$ code with $g(X) = 1 + X + X^3$:

- $g(X) \in \mathcal{C}$
- $Xg(X) = X + X^2 + X^4 \in \mathcal{C}$
- $(1 + X)g(X) = 1 + X^2 + X^3 + X^4 \in \mathcal{C}$

This illustrates both cyclic closure and absorption.

Example 10.2: Why Must $g(X)$ Divide $X^n - 1$?

For $g(X) = 1 + X + X^3$,

$$X^7 + 1 = (1 + X + X^3)(1 + X + X^2 + X^4) = g(X)h(X).$$

Thus $g(X)$ divides $X^7 + 1$, which is a necessary condition for any generator polynomial.

Number of Ideals in the Quotient Ring and Determination of the Generator Polynomial

From the previous section, cyclic codes correspond one-to-one with principal ideals of the quotient ring

$$R_n = \mathbb{F}_q[X]/\langle X^n - 1 \rangle.$$

This naturally leads to two questions:

1. How many ideals are there?
2. How is the generator polynomial that defines each ideal determined?

Main Theorem: Counting Ideals

Every ideal of R_n is a principal ideal generated by a monic divisor $g(X)$ of $X^n - 1$:

$$I = \langle g(X) \rangle.$$

Proof.

Consider the canonical projection

$$\pi : \mathbb{F}_q[X] \to R_n, \qquad f(X) \mapsto f(X) + \langle X^n - 1 \rangle.$$

This is a surjective ring homomorphism with kernel $\langle X^n - 1 \rangle$.

By the correspondence theorem for rings, there is a one-to-one correspondence between ideals I of R_n and ideals J of $\mathbb{F}_q[X]$ satisfying

$$\langle X^n - 1 \rangle \subseteq J.$$

Specifically,

$$I = J/\langle X^n - 1 \rangle, \qquad J = \pi^{-1}(I).$$

Since $\mathbb{F}_q[X]$ is a PID (indeed a Euclidean domain), every ideal J is principal:

$$J = \langle g(X) \rangle$$

for a unique monic polynomial $g(X)$.

The inclusion $\langle X^n - 1 \rangle \subseteq \langle g(X) \rangle$ is equivalent to

$$g(X) \mid (X^n - 1).$$

Conversely, every monic divisor $g(X)$ of $X^n - 1$ generates an ideal of R_n.

Therefore there is a one-to-one correspondence:

$$g(X) \mid (X^n - 1) \quad \longleftrightarrow \quad \langle g(X) \rangle \text{ is an ideal of } R_n.$$

■

Hence:

Number of ideals of R_n = number of monic divisors of $X^n - 1$ over $\mathbb{F}_q[X]$.

If $\gcd(n, q) = 1$, so that $X^n - 1$ has no repeated roots, and if

$$X^n - 1 = f_1(X) f_2(X) \cdots f_s(X)$$

is a product of s distinct irreducible polynomials, then each factor may be independently included or excluded from a divisor. Therefore the total number of monic divisors is

$$2^s.$$

This count includes both the whole ring ($g(X) = 1$) and the zero ideal ($g(X) = X^n - 1$).

Why 2^s? Every monic divisor corresponds uniquely to a subset $S \subseteq \{1, 2, \ldots, s\}$ through

$$g_S(X) = \prod_{i \in S} f_i(X).$$

Since the number of subsets is 2^s, the number of monic divisors is also 2^s.

Procedure for Determining Generator Polynomials

Step 1. Factor $X^n - 1$ completely into irreducible polynomials over $\mathbb{F}_q[X]$.

Step 2. Take products over all subsets of those irreducible factors.

Step 3. For each generator polynomial $g(X)$, the corresponding cyclic code has parameters

$$(n,\ n - \deg g(X)).$$

Example 13.1: Listing All Ideals for $n = 7$ over $\mathbb{F}_2$

Since

$$X^7 + 1 = (1 + X)(1 + X + X^3)(1 + X^2 + X^3),$$

there are $s = 3$ distinct irreducible factors, so the number of ideals is

$$2^3 = 8.$$

Subset S	$g(X) = \prod_{i \in S} f_i(X)$	$\deg(g)$	$[n, k]$	Number of codewords 2^k	Remark
$\emptyset$	1	0	$(7, 7)$	128	Whole ring (trivial)
$\{1\}$	$1 + X$	1	$(7, 6)$	64	Simple parity-check code
$\{2\}$	$1 + X + X^3$	3	$[7, 4]_2$	16	Hamming $(7, 4)$
$\{3\}$	$1 + X^2 + X^3$	3	$[7, 4]_2$	16	
$\{1, 2\}$	$1 + X^2 + X^3 + X^4$	4	$(7, 3)$	8	
$\{1, 3\}$	$1 + X + X^2 + X^4$	4	$(7, 3)$	8	
$\{2, 3\}$	$1 + X + X^2 + X^3 + X^4 + X^5 + X^6$	6	$(7, 1)$	2	Repetition code
$\{1, 2, 3\}$	$X^7 + 1$	7	$(7, 0)$	1	Zero ideal (trivial)

These 8 ideals form a lattice under inclusion: if

$$g_1(X) \mid g_2(X),$$

then

$$\langle g_2(X) \rangle \subseteq \langle g_1(X) \rangle.$$

So larger generator-polynomial degree means smaller code dimension and a smaller ideal.

Example 13.2: Number of Ideals for $n = 15$ over $\mathbb{F}_2$

From Exercise 2,

$$X^{15} + 1 = (1 + X)(1 + X + X^2)(1 + X + X^4)(1 + X^3 + X^4)(1 + X + X^2 + X^3 + X^4).$$

There are $s = 5$ irreducible factors, so the number of ideals is

$$2^5 = 32.$$

Among these 32 ideals, there are already several $(15, 11)$ cyclic codes, depending on which degree-4 factor or combination of factors is selected.

Example 13.3: Ideal Lattice and Inclusion Relations

For $n = 7$, the inclusion lattice looks like this:

```
        <1> = R7  (whole ring, k=7)
       / |  \
   <f1> <f2> <f3>   (k=6,4,4)
     |\ / \ /|
 <f1f2> <f1f3> <f2f3>  (k=3,3,1)
      \  |  /
     <f1f2f3> = <0>  (zero ideal, k=0)
```

As you move upward, the ideals get larger and contain more codewords. As you move downward, the ideals get smaller, the code dimension decreases, and the error-correcting capability generally improves.

Caution When $\gcd(n, q) \neq 1$

If $\gcd(n, q) \neq 1$, then $X^n - 1$ has repeated roots over $\mathbb{F}_q[X]$. For example, over $\mathbb{F}_2$,

$$X^6 + 1 = (1 + X)^2(1 + X + X^2)^2,$$

since $\gcd(6, 2) = 2$.

In that case, one may independently choose exponents

$$0 \leq e_i' \leq e_i$$

for each repeated factor $f_i(X)^{e_i}$, and the number of monic divisors becomes

$$\prod_{i=1}^{s}(e_i + 1).$$

For the example above, that is

$$(2 + 1)(2 + 1) = 9,$$

which is larger than $2^s = 4$.

Why Are Cyclic Codes Important in Practice?

The main practical strength of cyclic codes is that they admit **ultra-fast real-time hardware implementations** using LFSRs.

Principle of the LFSR Structure

Whereas a general linear block code requires large matrix multiplications, a cyclic code reduces to lightweight polynomial division implemented by:

- $r = \deg g(X)$ flip-flops,
- a few XOR gates matching the nonzero coefficients of the polynomial,
- and nothing more.

Advantage of Real-Time Processing

General linear code	Cyclic code (LFSR)
Buffer the whole block, then multiply by a matrix	Bit-by-bit real-time processing
Large memory required	Only r registers required
High latency	Near-zero latency
Complex parallel arithmetic	Simple serial XOR circuit

Real Example: CRC in Network Transmission

When checking the integrity of gigabit data streams in USB or Ethernet:

1. **No large memory is needed:** data can be processed as it arrives.
2. **On-the-fly computation:** one bit per clock is fed into the LFSR.
3. **Immediate result:** when the last bit passes, the remaining register contents are the parity bits or syndrome.

Example 11.1: LFSR Encoder for $g(X) = 1 + X + X^3$

For systematic encoding with $g(X) = 1 + X + X^3$, use three flip-flops $[R_0, R_1, R_2]$. The feedback is the XOR of the incoming bit and the most significant register R_2. Since the coefficients of X^0 and X^1 are 1, the feedback is XORed into R_0 and R_1.

Feed the message bits $(1, 0, 1, 1)$ in order from highest degree to lowest, i.e. m_3, m_2, m_1, m_0:

Clock	Input bit	Feedback $= \text{input} \oplus R_2$	R_0	R_1	R_2
Initial	-	-	0	0	0
1	$m_3 = 1$	$1 \oplus 0 = 1$	1	0	0
2	$m_2 = 1$	$1 \oplus 0 = 1$	1	1	0
3	$m_1 = 0$	$0 \oplus 0 = 0$	0	1	1
4	$m_0 = 1$	$1 \oplus 1 = 0$	0	0	1

After 4 clocks the register state is $(R_0, R_1, R_2) = (0, 0, 1)$.

> **Verification:** In Example 8.1, the remainder for systematic encoding of $m(X) = 1 + X^2 + X^3$ was $p(X) = 1$, i.e. parity vector $(1, 0, 0)$. Depending on whether the circuit is interpreted MSB-first or LSB-first, the register output order may be reversed. Reading $(0, 0, 1)$ backward gives $(1, 0, 0)$.
>
> **Remark:** The exact behavior of an LFSR depends on the feedback topology and the bit-input convention. The essential point is that polynomial division can be executed in real time using only $n - k$ registers and a few XOR gates.

What Are Representative Cyclic Codes?

Important code families with cyclic structure include:

Code	Characteristic	Main applications
CRC (Cyclic Redundancy Check)	Error detection only, many standard generator polynomials	Ethernet, USB, ZIP, storage media
BCH codes	Root-based construction guaranteeing error-correction capability	Flash memory, satellite communication
Reed-Solomon (RS) codes	Nonbinary cyclic code, symbol-level error correction	CD/DVD/Blu-ray, QR codes, deep-space communication
Some Hamming codes	Cyclic form for certain parameters	ECC memory

Example 15.1: CRC-8 Generator Polynomial

A standard CRC-8 polynomial is

$$g(X) = X^8 + X^2 + X + 1.$$

- Code length: variable (depends on the message length)
- Parity length: 8 bits
- Typical use: sensor networks and embedded systems

Example 15.2: CRC-32 Used in Ethernet

$$g(X) = X^{32}+X^{26}+X^{23}+X^{22}+X^{16}+X^{12}+X^{11}+X^{10}+X^8+X^7+X^5+X^4+X^2+X+1.$$

- 32 parity bits
- Protects frames up to length $2^{32} - 1$ in theory
- Used for the FCS (Frame Check Sequence) in Ethernet

What Are Quasi-Cyclic Codes and the Post-Quantum Cryptosystem HQC?

The algebraic structure of cyclic codes plays an important role in modern cryptography, especially in **post-quantum cryptography (PQC)**.

Quasi-Cyclic Codes (QC)

If a cyclic code is closed under a 1-step shift, then a **quasi-cyclic code** is closed under an ℓ-step shift.

- If the code length is $n = m\ell$, then shifting any codeword by ℓ positions preserves membership in the code.
- If $\ell = 1$, we recover an ordinary cyclic code.
- Generator and parity-check matrices are built from $m \times m$ **circulant blocks**.

Use in HQC (Hamming Quasi-Cyclic)

HQC is a code-based key encapsulation mechanism designed to remain secure against quantum computers, including attacks based on Shor's algorithm. It uses quasi-cyclic structure in a central way.

The classical problem: the McEliece cryptosystem is highly secure but suffers from very large public keys, typically around the megabyte scale.

The HQC idea: adopt a quasi-cyclic structure in order to obtain:

1. **Smaller public keys:** a circulant matrix is determined entirely by its first row, so one polynomial can represent a full matrix block.
2. **Faster arithmetic:** operations are performed in a polynomial ring such as $\mathbb{F}_2[X]/\langle X^n - 1\rangle$, enabling efficient algorithms.
3. **Provable security reductions:** the scheme is related to hard syndrome-decoding problems believed to remain difficult even for quantum computers.

Example 16.1: Structure of a Circulant Block

Consider the 4×4 circulant matrix whose first row is $(1, 0, 1, 1)$:

$$C = \begin{bmatrix} 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 \\ 1 & 1 & 1 & 0 \\ 0 & 1 & 1 & 1 \end{bmatrix}.$$

Each row is obtained from the previous one by a one-step cyclic right shift. Therefore storing only the first row is enough to reconstruct the entire matrix. This is precisely why quasi-cyclic structure can reduce public-key size so dramatically.

Exercises

Exercise 1. Determining Whether a Code Is Cyclic

Decide whether the following code is cyclic. ($n = 5$)

$$\mathcal{C} = \{00000,\ 10110,\ 01011,\ 11101,\ 01101,\ 10011,\ 00111,\ 11010,\ 11001,\ 00101, \dots\}$$

Hint: apply a one-step cyclic shift to each codeword and check whether the result is still in the set.

Exercise 2. Finding Generator Polynomials

Suppose that over $GF(2)$,

$$X^{15} + 1 = (1 + X)(1 + X + X^2)(1 + X + X^4)(1 + X^3 + X^4)(1 + X + X^2 + X^3 + X^4).$$

(a) List all degree-4 generator polynomials that can define a $(15, 11)$ cyclic code.

(b) If $g(X) = 1 + X + X^4$, compute the corresponding parity-check polynomial $h(X)$.

Exercise 3. Practice with Systematic Encoding

For the $[7,4]_2$ cyclic code with $g(X) = 1 + X + X^3$, systematically encode the following messages:

(a) $\mathbf{m} = (0,1,1,0)$

(b) $\mathbf{m} = (1,1,1,1)$

(c) Check that your results agree with the codeword table in Example 8.4.

Exercise 4. Syndrome Computation and Error Correction

For the $[7,4]_2$ cyclic code, compute the syndrome of each received vector and correct it if it contains a single-bit error:

(a) $\mathbf{v} = (1,1,0,1,0,0,0)$

(b) $\mathbf{v} = (0,1,1,0,1,0,1)$

(c) $\mathbf{v} = (1,0,1,1,1,0,0)$

Hint: use the syndrome table of Example 9.3.

Exercise 5. Generator and Parity-Check Matrices

Consider the $[7,4]_2$ cyclic code with generator polynomial

$$g(X) = 1 + X^2 + X^3.$$

(a) Find the non-systematic generator matrix G.

(b) Compute the parity-check polynomial $h(X)$.

(c) Construct the parity-check matrix H.

(d) Verify that $GH^T = 0$.

Exercise 6. Understanding the Algebraic Structure

(a) Factor $X^6 + 1$ over $GF(2)$.

Caution: since $\gcd(6,2) = 2 \neq 1$, the polynomial $X^6 + 1$ has repeated roots over $GF(2)$, so some standard results of cyclic-code theory do not apply in their usual form. Confirm from the factorization that repeated factors occur.

(b) Based on that factorization, list the divisors of $X^6 + 1$ and determine the corresponding $[n,k]$ parameters.

(c) Find the generator polynomial of the code with largest dimension, and determine its minimum distance.

Exercise 7. Comprehensive Problem

Let the generator polynomial of a $(15, 7)$ BCH code be

$$g(X) = 1 + X^4 + X^6 + X^7 + X^8.$$

(a) Verify that this is a cyclic code by checking that $g(X)$ divides $X^{15} + 1$.

(b) Systematically encode the message

$$\mathbf{m} = (1, 0, 0, 0, 0, 0, 1).$$

(c) Verify that the syndrome of the encoded codeword is 0.

SageMath Practice Code

Running the following SageMath script allows you to verify the main concepts discussed in this document directly.

```
# ============================================================
# Cyclic Codes - SageMath practice code
# Verifies the examples discussed in this document.
# ============================================================

F = GF(2)
R.<X> = PolynomialRing(F)

# ============================================================
# 1. Factorization of X^7 + 1 (Example 3.1)
# ============================================================
print("=" * 70)
print("1. Factorization of X^7 + 1 over GF(2)")
print("=" * 70)

f = X^7 + 1
print(f"X^7 + 1 = {factor(f)}")
print()

# Three irreducible polynomials
p1 = 1 + X
p2 = 1 + X + X^3
p3 = 1 + X^2 + X^3

print("Check of irreducible factorization:")
print(f"  (1+X)(1+X+X^3)(1+X^2+X^3) = {p1 * p2 * p3}")
print(f"  Matches X^7 + 1: {'✓' if p1 * p2 * p3 == f else '✗'}")
print()

# List all possible generator polynomials
print("Possible generator polynomials g(X) and corresponding [n, k]:")
print("-" * 50)
irreducibles = [p1, p2, p3]
from itertools import combinations
all_divisors = [R(1)]
```

```
for r in range(1, 4):
    for combo in combinations(irreducibles, r):
        prod = R(1)
        for p in combo:
            prod *= p
        all_divisors.append(prod)
all_divisors.append(f)
all_divisors.sort(key=lambda p: p.degree())

for g in all_divisors:
    deg = g.degree()
    k = 7 - deg
    if 0 <= k <= 7:
        print(f"  g(X) = {g},  deg = {deg},  (7, {k}),  number of codewords = {2^k}")
print()

# ============================================================
# 2. Constructing the (7,4) cyclic code - g(X) = 1 + X + X^3
# ============================================================
print("=" * 70)
print("2. [7, 4]_2 cyclic code: g(X) = 1 + X + X^3")
print("=" * 70)

g = 1 + X + X^3
n = 7
k = n - g.degree()

C = codes.CyclicCode(generator_pol=g, length=n)
print(f"Code parameters: [{C.length()}, {C.dimension()}, {C.minimum_distance()}]")
print(f"Generator polynomial: g(X) = {g}")
print()

# Parity-check polynomial (Example 4.1)
h = (X^7 + 1) // g
print(f"Parity-check polynomial: h(X) = {h}")
print(f"Verification: g(X) * h(X) = {g * h}  (== X^7+1: {'✓' if g * h == f else 'x'})")
print()

# ============================================================
# 3. Generator matrix and parity-check matrix
# ============================================================
print("=" * 70)
print("3. Generator matrix G and parity-check matrix H")
print("=" * 70)

G = Matrix(F, k, n)
for i in range(k):
    poly = (X^i * g) % (X^n + 1)
    for j in range(n):
        G[i, j] = poly[j]

print("Non-systematic generator matrix G:")
print(G)
print()
```

```
h_coeffs = [h[k - j] for j in range(k + 1)]  # reversed coefficients
H = Matrix(F, n - k, n)
for i in range(n - k):
    for j in range(k + 1):
        H[i, i + j] = h_coeffs[j]

print("Parity-check matrix H:")
print(H)
print()

product = G * H.transpose()
print(f"Verification that G * H^T = 0: {'✓' if product == 0 else '✗'}")
print(G * H.transpose())
print()

# ============================================================
# 4. Non-systematic encoding (Examples 3.2 and 3.3)
# ============================================================
print("=" * 70)
print("4. Non-systematic encoding: c(X) = m(X) * g(X)")
print("=" * 70)

test_messages_nonsys = [
    ("(1,1,0,1)", 1 + X + X^3),
    ("(0,1,0,0)", X),
    ("(1,0,1,1)", 1 + X^2 + X^3),
]

for label, m_poly in test_messages_nonsys:
    c_poly = (m_poly * g) % (X^n + 1)
    c_vec = vector(F, [c_poly[i] for i in range(n)])
    print(f"  m = {label},  m(X) = {m_poly}")
    print(f"  c(X) = {c_poly},  c = {list(c_vec)}")
    remainder = c_poly % g
    print(f"  c(X) mod g(X) = {remainder}  ({'✓ multiple of g(X)' if remainder == 0 else '✗'})")
    print()

# ============================================================
# 5. Systematic encoding (Examples 8.1-8.3)
# ============================================================
print("=" * 70)
print("5. Systematic encoding: c(X) = X^(n-k)*m(X) + [X^(n-k)*m(X) mod g(X)]")
print("=" * 70)

def systematic_encode(m_poly, g, n, k):
    """Perform systematic encoding."""
    r = n - k
    shifted = X^r * m_poly
    _, p = shifted.quo_rem(g)
    c_poly = shifted + p
    c_vec = vector(F, [c_poly[i] for i in range(n)])
    return c_poly, p, c_vec
```

```
test_messages = [
    ("(1,0,1,1)", 1 + X^2 + X^3),
    ("(1,1,0,0)", 1 + X),
    ("(0,0,0,1)", X^3),
    ("(0,1,1,0)", X + X^2),
    ("(1,1,1,1)", 1 + X + X^2 + X^3),
]

for label, m_poly in test_messages:
    c_poly, p, c_vec = systematic_encode(m_poly, g, n, k)
    p_vec = vector(F, [p[i] for i in range(n - k)])
    m_vec = vector(F, [m_poly[i] for i in range(k)])
    print(f"  m = {label},  m(X) = {m_poly}")
    print(f"  parity p(X) = {p},  p = {list(p_vec)}")
    print(f"  codeword c = {list(c_vec)}")
    print(f"  structure [parity|message] = {list(p_vec)}|{list(m_vec)}")
    remainder = c_poly % g
    print(f"  c(X) mod g(X) = {remainder}  ({'✓' if remainder == 0 else '✗'})")
    print()

# ============================================================
# 6. Full systematic codeword table (Example 8.4)
# ============================================================
print("=" * 70)
print("6. Full systematic codeword table for the (7,4) code (16 codewords)")
print("=" * 70)
print(f"{'message':<12} {'m(X)':<20} {'parity':<10} {'codeword':<16} {'wH':>4}")
print("-" * 65)

V4 = VectorSpace(F, 4)
codeword_set = set()

for m_vec in V4:
    m_poly = sum(F(m_vec[i]) * X^i for i in range(k))
    c_poly, p, c_vec = systematic_encode(m_poly, g, n, k)
    p_vec = [p[i] for i in range(n - k)]
    wH = sum(int(c_vec[i]) for i in range(n))
    codeword_set.add(tuple(c_vec))
    print(f"  {list(m_vec)!s:<12} {str(m_poly):<20} {str(p_vec):<10} {list(c_vec)!s:<16} {wH:>4

print(f"\nTotal number of codewords: {len(codeword_set)} (expected: {2^k})")
print()

# ============================================================
# 7. Verification of cyclic-shift invariance
# ============================================================
print("=" * 70)
print("7. Verification of cyclic-shift invariance")
print("=" * 70)

def cyclic_shift(v, n):
    """Shift vector v one step cyclically to the right."""
    v = list(v)
    return tuple([v[-1]] + v[:-1])
```

```
cyclic_ok = True
for cw in codeword_set:
    shifted = cw
    for s in range(1, n):
        shifted = cyclic_shift(shifted, n)
        if shifted not in codeword_set:
            print(f"  ✗ {cw} -> shift by {s} -> {shifted} : not a codeword!")
            cyclic_ok = False

if cyclic_ok:
    print("  ✓ Every cyclic shift of every codeword is still in the code.")
    print("  -> Confirmed: this is a cyclic code.")
print()

# ============================================================
# 8. Syndrome table (Example 9.3)
# ============================================================
print("=" * 70)
print("8. Syndrome table for single-bit errors")
print("=" * 70)
print(f"{'error position i':<16} {'e(X)':<12} {'s(X) = e(X) mod g(X)':<24} {'syndrome vector'}"
print("-" * 70)

syndrome_table = {}
for i in range(n):
    e_poly = X^i
    s_poly = e_poly % g
    s_vec = tuple(F(s_poly[j]) for j in range(n - k))
    syndrome_table[s_vec] = i
    print(f"  {i:<16} {str(e_poly):<12} {str(s_poly):<24} {s_vec}")

print(f"\nNumber of distinct syndromes: {len(syndrome_table)} (expected: {n})")
all_distinct = len(syndrome_table) == n
print(f"All single-bit errors are correctable: {'✓' if all_distinct else '✗'}")
print()

# ============================================================
# 9. Syndrome-based error-correction simulation (Example 9.4)
# ============================================================
print("=" * 70)
print("9. Syndrome-based error detection and correction simulation")
print("=" * 70)

m_orig = 1 + X^2 + X^3  # m = (1,0,1,1)
c_poly_orig, _, c_vec_orig = systematic_encode(m_orig, g, n, k)
print(f"Original message: m = (1,0,1,1)")
print(f"Codeword:         c = {list(c_vec_orig)}")
print()

print("Inject a single-bit error -> compute syndrome -> correct:")
print("-" * 70)

for err_pos in range(n):
```

```
    e_poly = X^err_pos
    v_poly = c_poly_orig + e_poly
    v_vec = vector(F, [v_poly[i] for i in range(n)])

    s_poly = v_poly % g
    s_vec = tuple(F(s_poly[j]) for j in range(n - k))

    if s_vec in syndrome_table:
        est_pos = syndrome_table[s_vec]
        corrected_poly = v_poly + X^est_pos
        corrected_vec = vector(F, [corrected_poly[i] for i in range(n)])
        success = (corrected_vec == c_vec_orig)
        print(f"  error position={err_pos}, received={list(v_vec)}, "
              f"syndrome={s_vec}, estimated position={est_pos}, "
              f"corrected={'✓' if success else '✗'}")
    else:
        print(f"  error position={err_pos}, syndrome={s_vec}, not in table!")

print()

# ============================================================
# 10. Miscorrection under a 2-bit error (Example 9.5)
# ============================================================
print("=" * 70)
print("10. Miscorrection under a 2-bit error")
print("=" * 70)

e_poly_2bit = 1 + X
v_poly_2bit = c_poly_orig + e_poly_2bit
v_vec_2bit = vector(F, [v_poly_2bit[i] for i in range(n)])
s_poly_2bit = v_poly_2bit % g
s_vec_2bit = tuple(F(s_poly_2bit[j]) for j in range(n - k))

print(f"Original codeword: {list(c_vec_orig)}")
print(f"Error pattern:     e(X) = 1 + X (positions 0 and 1)")
print(f"Received vector:   {list(v_vec_2bit)}")
print(f"Syndrome:          {s_vec_2bit}")

if s_vec_2bit in syndrome_table:
    est_pos = syndrome_table[s_vec_2bit]
    corrected = v_poly_2bit + X^est_pos
    corrected_vec = vector(F, [corrected[i] for i in range(n)])
    is_correct = (corrected_vec == c_vec_orig)
    print(f"Estimated error position: {est_pos} (actual positions: 0 and 1)")
    print(f"Correction result: {list(corrected_vec)}")
    print(f"Successful correction: {'✓' if is_correct else '✗ miscorrection occurred!'}")
print()

# ============================================================
# 11. Codeword validation using the parity-check polynomial
# ============================================================
print("=" * 70)
print("11. Validating codewords with h(X): c(X)*h(X) ≡ 0 mod (X^7+1)")
print("=" * 70)
```

```
h = (X^7 + 1) // g
print(f"h(X) = {h}")
print()

all_valid = True
for cw in codeword_set:
    c_poly_check = sum(F(cw[i]) * X^i for i in range(n))
    product = (c_poly_check * h) % (X^n + 1)
    if product != 0:
        print(f"  ✗ c = {cw}: c(X)*h(X) mod (X^7+1) = {product}")
        all_valid = False

if all_valid:
    print(f"  ✓ For all {len(codeword_set)} codewords, c(X)*h(X) ≡ 0 (mod X^7+1)")
print()

# ============================================================
# 12. Cyclic codes generated by different divisors of X^7+1
# ============================================================
print("=" * 70)
print("12. Cyclic codes generated by various divisors of X^7+1")
print("=" * 70)

generators = [
    ("1+X",                   1 + X),
    ("1+X+X^3",               1 + X + X^3),
    ("1+X^2+X^3",             1 + X^2 + X^3),
    ("(1+X)(1+X+X^3)",        (1 + X) * (1 + X + X^3)),
    ("(1+X)(1+X^2+X^3)",      (1 + X) * (1 + X^2 + X^3)),
    ("(1+X+X^3)(1+X^2+X^3)",  (1 + X + X^3) * (1 + X^2 + X^3)),
]

print(f"{'g(X)':<30} {'deg':<6} {'(n,k)':<10} {'d_min':<8} {'# codewords'}")
print("-" * 70)

for label, gen in generators:
    try:
        C_test = codes.CyclicCode(generator_pol=gen, length=7)
        d_min = C_test.minimum_distance()
        dim = C_test.dimension()
        print(f"  {label:<30} {gen.degree():<6} (7,{dim}){'':<5} {d_min:<8} {2^dim}")
    except Exception as e:
        print(f"  {label:<30} error: {e}")

print()

# ============================================================
# 13. LFSR simulation for systematic encoding
# ============================================================
print("=" * 70)
print("13. LFSR simulation: systematic encoding with g(X)=1+X+X^3")
print("=" * 70)
```

```
def lfsr_systematic_encode(message_bits, g_coeffs, n, k):
    """
    Simulate systematic encoding using an LFSR.
    message_bits: list of k message bits [m_0, m_1, ..., m_{k-1}]
    g_coeffs: coefficients [g_0, g_1, ..., g_{n-k-1}]
    Bits are fed in from highest degree to lowest.
    """
    r = n - k
    reg = [F(0)] * r

    print(f"  {'clock':<6} {'input':<8} {'feedback':<10} {'registers'}")
    print(f"  {'init':<6} {'-':<8} {'-':<10} {reg}")

    for clk in range(k):
        input_bit = F(message_bits[k - 1 - clk])
        feedback = input_bit + reg[r - 1]

        new_reg = [F(0)] * r
        new_reg[0] = feedback * F(g_coeffs[0])
        for j in range(1, r):
            new_reg[j] = reg[j - 1] + feedback * F(g_coeffs[j])

        reg = new_reg
        print(f"  {clk+1:<6} {int(input_bit):<8} {int(feedback):<10} {[int(x) for x in reg]}")

    return [int(x) for x in reg]

g_coeffs = [1, 1, 0]
message = [1, 0, 1, 1]

print(f"Message: {message}")
print(f"g(X) = 1 + X + X^3, feedback coefficients: g_0={g_coeffs[0]}, g_1={g_coeffs[1]}, g_2={g
print()

parity_lfsr = lfsr_systematic_encode(message, g_coeffs, n, k)
print(f"\nLFSR output (register state): {parity_lfsr}")

m_poly_check = 1 + X^2 + X^3
_, p_check = (X^3 * m_poly_check).quo_rem(g)
p_check_vec = [int(p_check[i]) for i in range(3)]
print(f"Parity from polynomial division: {p_check_vec}")
print()

# ============================================================
# 14. Example of a (7,3) cyclic code (Example 5.3)
# ============================================================
print("=" * 70)
print("14. (7,3) cyclic code: g(X) = 1 + X^2 + X^3 + X^4")
print("=" * 70)

g73 = 1 + X^2 + X^3 + X^4
C73 = codes.CyclicCode(generator_pol=g73, length=7)
print(f"Code parameters: [{C73.length()}, {C73.dimension()}, {C73.minimum_distance()}]")
print(f"Generator polynomial: g(X) = {g73}")
```

```
print()

G73 = Matrix(F, 3, 7)
for i in range(3):
    poly = (X^i * g73) % (X^7 + 1)
    for j in range(7):
        G73[i, j] = poly[j]

print("Generator matrix G:")
print(G73)
print()

m73 = vector(F, [1, 1, 0])
c73 = m73 * G73
print(f"m = (1,1,0) -> c = {list(c73)}")
print()

# ============================================================
# 15. Comparison with SageMath's built-in CyclicCode
# ============================================================
print("=" * 70)
print("15. Comparing manual computation with SageMath's built-in CyclicCode")
print("=" * 70)

C_builtin = codes.CyclicCode(generator_pol=g, length=7)
G_builtin = C_builtin.systematic_generator_matrix()
print("Systematic generator matrix from SageMath:")
print(G_builtin)
print()

match_count = 0
total = 2^k
for m_vec in VectorSpace(F, k):
    c_builtin = m_vec * G_builtin
    m_poly = sum(F(m_vec[i]) * X^i for i in range(k))
    _, _, c_manual = systematic_encode(m_poly, g, n, k)

    if list(c_builtin) == list(c_manual):
        match_count += 1

print(f"Matching codewords: {match_count}/{total}")
print(f"Manual encoding and SageMath built-in result {'match perfectly ✓' if match_count == tot
print()

# ============================================================
# 16. Structure of a circulant block (Example 16.1)
# ============================================================
print("=" * 70)
print("16. Circulant-matrix block structure")
print("=" * 70)

def circulant_matrix(first_row, field=F):
    """Construct a circulant matrix from its first row."""
    m = len(first_row)
```

```
    M = Matrix(field, m, m)
    for i in range(m):
        for j in range(m):
            M[i, j] = first_row[(j - i) % m]
    return M

first_row = [F(1), F(0), F(1), F(1)]
C_circ = circulant_matrix(first_row)
print(f"First row: {first_row}")
print("Circulant matrix:")
print(C_circ)
print()

circ_ok = True
for i in range(1, 4):
    prev_row = list(C_circ[i - 1])
    expected = [prev_row[-1]] + prev_row[:-1]
    actual = list(C_circ[i])
    if actual != expected:
        circ_ok = False
        print(f"  ✗ row {i}: expected {expected}, got {actual}")

if circ_ok:
    print("  ✓ Every row is a cyclic right shift of the previous row.")
print()

print("=" * 70)
print("All checks completed!")
print("=" * 70)
```

References

[1] C. E. Shannon, "A Mathematical Theory of Communication," *Bell System Technical Journal*, vol. 27, no. 3, pp. 379-423, 1948.

[2] R. W. Hamming, "Error Detecting and Error Correcting Codes," *Bell System Technical Journal*, vol. 29, no. 2, pp. 147-160, 1950.

[3] D. E. Muller, "Application of Boolean Algebra to Switching Circuit Design and to Error Detection," *IRE Transactions on Electronic Computers*, vol. EC-3, no. 3, pp. 6-12, 1954.

[4] I. S. Reed, "A Class of Multiple-Error-Correcting Codes and the Decoding Scheme," *IRE Transactions on Information Theory*, vol. 4, no. 4, pp. 38-49, 1954.

[5] E. Prange, "Cyclic Error-Correcting Codes in Two Symbols," *AFCRC-TN-57-103*, Air Force Cambridge Research Center, 1957.

[6] A. Hocquenghem, "Codes correcteurs d'erreurs," *Chiffres*, vol. 2, pp. 147-156, 1959.

[7] R. C. Bose and D. K. Ray-Chaudhuri, "On a Class of Error Correcting Binary Group Codes," *Information and Control*, vol. 3, no. 1, pp. 68-79, 1960.

[8] I. S. Reed and G. Solomon, "Polynomial Codes Over Certain Finite Fields," *Journal of the Society for Industrial and Applied Mathematics*, vol. 8, no. 2, pp. 300-304, 1960.

[9] W. W. Peterson, *Error-Correcting Codes*, MIT Press, 1961.

[10] E. R. Berlekamp, *Algebraic Coding Theory*, McGraw-Hill, 1968.

[11] J. L. Massey, “Shift-Register Synthesis and BCH Decoding,” *IEEE Transactions on Information Theory*, vol. 15, no. 1, pp. 122-127, 1969.

[12] V. D. Goppa, “Geometry and Codes,” *Mathematics and its Applications*, vol. 24, Kluwer, 1988.

[13] R. J. McEliece, “A Public-Key Cryptosystem Based on Algebraic Coding Theory,” *DSN Progress Report*, Jet Propulsion Laboratory, 1978.

[14] C. Berrou, A. Glavieux, and P. Thitimajshima, “Near Shannon Limit Error-Correcting Coding and Decoding: Turbo Codes,” in *Proceedings of ICC ’93*, 1993.

[15] D. J. C. MacKay and R. M. Neal, “Near Shannon Limit Performance of Low Density Parity Check Codes,” *Electronics Letters*, vol. 32, no. 18, pp. 1645-1646, 1996.